

Chyby testování a ladění programu

Chyby v programu

- **chyba v programu** = situace, kdy program nefunguje správně
- chyba může vzniknout:
 - při psaní kódu
 - při překladu programu
 - při běhu programu
 - kvůli špatnému algoritmu
 - kvůli neočekávanému vstupu
 - kvůli prostředí, ve kterém program běží

Příklady:

- překlep v kódu
- špatná podmínka
- dělení nulou
- neexistující soubor
- špatný vstup od uživatele
- výpadek internetu

Druhy chyb

Syntaktické chyby

- vznikají porušením pravidel jazyka
- program nejde správně přeložit nebo spustit
- odhalí je kompilátor, překladač nebo editor
- často se označují jako **compile-time errors**

Příklady:

- chybějící středník
- chybějící závorka
- špatně napsané klíčové slovo
- neuzavřený string

Příklad:

```
Console.WriteLine("Ahoj")
```

- chybí uzavírací závorka)

Typové chyby

- vznikají při práci s nesprávným datovým typem
- ve staticky typovaných jazycích je často odhalí kompilátor
- v dynamicky typovaných jazycích se mohou projevit až za běhu

Příklad v C#:

```
int cislo = "ahoj";
```

- do `int` nejde uložit text

Poznámka:

- `var` v C# neznamená dynamický typ
- typ se odvodí při kompilaci
- `dynamic` obchází část typové kontroly a chyby se mohou projevit až za běhu

Sémantické / logické chyby

- program jde spustit
- program nepadá
- výsledek je ale špatný
- problém je v logice programu

Příklady:

- špatný vzorec
- špatná podmínka
- nekonečný cyklus
- chyba o jeden index
- špatně navržený algoritmus

Příklad:

```
int prumer = soucet / pocet + 1;
```

- program může běžet
- ale výpočet průměru je logicky špatně

Běhové chyby

- vznikají až při běhu programu

- program může být syntakticky správný
- chyba nastane kvůli konkrétní situaci
- často se řeší pomocí výjimek
- označují se jako **runtime errors**

Příklady:

- dělení nulou
- soubor neexistuje
- chybí oprávnění
- disk je plný
- vypadne internet
- uživatel zadá text místo čísla
- přístup mimo rozsah pole

Příklad:

```
int[] cisla = { 1, 2, 3 };  
Console.WriteLine(cisla[5]);
```

- pole nemá index 5
- vznikne běhová chyba

Výjimky

Co je výjimka

- **výjimka** = objekt popisující chybu, která nastala za běhu programu
- v C# vychází z třídy `Exception`
- když chyba nastane, program výjimku „vyhodí“
- pokud ji nikdo nezachytí, program obvykle spadne

Příklady výjimek:

- `Exception`
- `FileNotFoundException`
- `IOException`
- `FormatException`
- `DivideByZeroException`
- `IndexOutOfRangeException`
- `NullReferenceException`
- `UnauthorizedAccessException`

Call stack

- **call stack** = zásobník volání funkcí
- ukazuje, jaké metody byly volány
- když vznikne výjimka, postupuje zpět přes call stack
- pokud ji žádný `catch` nezachytí, program skončí chybou

Zjednodušeně:

```
Main()
  → NactiData()
    → OtevriSoubor()
      → chyba
```

Ošetřování výjimek

`try`

- blok kódu, kde může nastat chyba
- program se pokusí tento kód vykonat

`catch`

- blok, který se spustí, pokud v `try` nastane výjimka
- slouží k zachycení a zpracování chyby

`finally`

- blok, který se provede vždy
- provede se při úspěchu i při chybě
- používá se například pro úklid prostředků

Příklad:

```
try
{
    int cislo = int.Parse("abc");
    Console.WriteLine(cislo);
}
catch (FormatException)
{
    Console.WriteLine("Text nelze převést na číslo.");
}
finally
{
    Console.WriteLine("Hotovo.");
}
```

Více catch bloků

- různé chyby můžeme řešit různě
- konkrétnější výjimky se dávají před obecnější

Příklad:

```
try
{
    string text = File.ReadAllText("data.txt");
    Console.WriteLine(text);
}
catch (FileNotFoundException)
{
    Console.WriteLine("Soubor nebyl nalezen.");
}
catch (UnauthorizedAccessException)
{
    Console.WriteLine("Nemáš oprávnění k souboru.");
}
catch (IOException)
{
    Console.WriteLine("Chyba při práci se souborem.");
}
catch (Exception)
{
    Console.WriteLine("Nastala obecná chyba.");
}
```

throw

- slouží k ručnímu vyhození výjimky
- používá se, když metoda dostane neplatná data
- nebo když nelze pokračovat dál

Příklad:

```
void NastavVek(int vek)
{
    if (vek < 0)
    {
        throw new ArgumentException("Věk nesmí být záporný.");
    }
}
```

Defenzivní programování

- snaha chybám předcházet
- program kontroluje vstupy a stav před provedením akce
- cílem je snížit riziko chyb

Příklady:

```
if (File.Exists(cesta))
{
    string text = File.ReadAllText(cesta);
}
```

```
if (pocet != 0)
{
    int vysledek = soucet / pocet;
}
```

Race condition

- situace, kdy se stav změní mezi kontrolou a akcí
- i po kontrole může chyba nastat
- proto kontrola často nestačí a je vhodné používat i `try-catch`

Příklad:

- program zkontroluje, že soubor existuje
- jiný proces soubor mezitím smaže
- program se ho pokusí otevřít
- vznikne chyba

Mechanismy hlášení chyb

Návratové kódy

- starší způsob oznamování chyb
- funkce vrátí hodnotu podle výsledku
- například:
 - `0` = úspěch
 - `-1` = chyba

Příklad:

```
int vysledek = UlozData();

if (vysledek == -1)
{
```

```
Console.WriteLine("Chyba při ukládání.");  
}
```

Nevýhody:

- programátor musí návratovou hodnotu kontrolovat
- snadno se na kontrolu zapomene
- kód může být méně přehledný

Výjimky

- modernější způsob hlášení chyb
- chyba se předá jako objekt
- výjimka může obsahovat:
 - typ chyby
 - zprávu
 - call stack
 - další data

Výhody:

- oddělují běžný kód od řešení chyb
- lze zachytit různé typy chyb
- nelze je tak snadno ignorovat jako návratové kódy

Ladění programu

Co je ladění

- **ladění** = hledání a opravování chyb v programu
- cílem je zjistit:
 - kde chyba vzniká
 - proč vzniká
 - jak ji opravit

Výpisy do konzole

- jednoduchý způsob ladění
- používá se například:
 - `Console.WriteLine`
 - `console.log`
- vhodné pro rychlou kontrolu hodnot

Příklad:

```
Console.WriteLine("Hodnota x: " + x);
```

Nevýhoda:

- u větších programů může být nepřehledné
- po ladění je potřeba výpisy odstranit nebo nahradit logováním

Debugger

- nástroj pro ladění programu
- umožňuje zastavit program a sledovat jeho stav
- je součástí vývojových prostředí

Příklady IDE:

- Visual Studio
- Rider
- VS Code
- IntelliJ IDEA

Breakpoint

- místo v kódu, kde se program při ladění zastaví
- umožní zkontrolovat aktuální stav programu

Watch

- okno, kde sledujeme hodnoty proměnných
- můžeme vidět, jak se hodnoty mění během běhu programu

Krokování

Step Over

- provede aktuální řádek
- pokud je na řádku volání metody, nevstoupí dovnitř
- přejde na další řádek

Step Into

- vstoupí dovnitř volané metody
- používá se, když chceme sledovat průběh uvnitř funkce

Step Out

- dokončí aktuální metodu

- vrátí se zpět na místo, odkud byla metoda zavolána

Testování programu

Co je testování

- **testování** = ověřování, že program funguje správně
- cílem je najít chyby dříve, než se dostanou k uživateli
- testuje se:
 - správný vstup
 - špatný vstup
 - hraniční případy
 - očekávané výsledky

Edge cases

- **edge cases** = hraniční případy
- vstupy, které mohou odhalit chybu
- často nejsou úplně běžné, ale jsou důležité

Příklady:

- prázdný vstup
- `null`
- záporné číslo
- nula
- velmi velké číslo
- pole bez prvků
- jeden prvek v poli
- neexistující soubor

Druhy testů

Unit testy

- jednotkové testy
- testují malou část programu
- typicky jednu metodu nebo funkci
- testují se izolovaně
- mají být rychlé a opakovatelné

Příklad:

```
int Secti(int a, int b)
{
    return a + b;
}
```

Testujeme například:

- `Secti(2, 3)` vrátí 5
- `Secti(-1, 1)` vrátí 0
- `Secti(0, 0)` vrátí 0

Mockování

- používá se při testování
- nahrazuje skutečnou závislost falešnou verzí
- hodí se například pro:
 - databázi
 - API
 - souborový systém
 - e-mailovou službu

Příklad:

- metoda potřebuje databázi
- v testu nechceme používat skutečnou databázi
- použijeme falešnou databázi

Integrační testy

- testují spolupráci více částí programu
- ověřují, že jednotlivé moduly správně spolupracují
- například:
 - aplikace komunikuje s databází
 - API vrací správná data
 - služby si správně předávají data

End-to-end testy

- zkratka **E2E**
- testují celý průchod aplikací
- simulují reálného uživatele
- často se používají u webových aplikací

Příklad:

- uživatel se přihlásí
- přidá zboží do košíku
- odešle objednávku
- systém zobrazí potvrzení

Nástroje:

- Playwright
- Cypress
- Selenium

Test renderování komponent

- používá se hlavně u frontendových aplikací
- ověřuje, že se komponenta správně zobrazí
- může testovat:
 - text na stránce
 - tlačítka
 - vstupní pole
 - reakci na kliknutí

Praktické ukázky

1. Ošetření převodu textu na číslo

```
Console.WriteLine("Zadej číslo:");
string vstup = Console.ReadLine();

try
{
    int cislo = int.Parse(vstup);
    Console.WriteLine("Zadal jsi: " + cislo);
}
catch (FormatException)
{
    Console.WriteLine("Tohle není platné číslo.");
}
```

2. Bezpečnější převod pomocí TryParse

```
Console.WriteLine("Zadej číslo:");
string vstup = Console.ReadLine();

if (int.TryParse(vstup, out int cislo))
{
    Console.WriteLine("Zadal jsi: " + cislo);
}
```

```
        Console.WriteLine("Zadal jsi: " + cislo);
    }
    else
    {
        Console.WriteLine("Tohle není platné číslo.");
    }
}
```

3. Dělení s kontrolou nuly

```
int a = 10;
int b = 0;

if (b != 0)
{
    Console.WriteLine(a / b);
}
else
{
    Console.WriteLine("Nelze dělit nulou.");
}
```

4. Práce se souborem s try-catch

```
try
{
    string text = File.ReadAllText("data.txt");
    Console.WriteLine(text);
}
catch (FileNotFoundException)
{
    Console.WriteLine("Soubor nebyl nalezen.");
}
catch (IOException)
{
    Console.WriteLine("Chyba při čtení souboru.");
}
```

5. Jednoduchý unit test

Příklad metody:

```
int Secti(int a, int b)
{
    return a + b;
}
```

Příklad testu:

```
[Test]
public void Secti_2A3_Vrati5()
{
    int vysledek = Secti(2, 3);

    Assert.AreEqual(5, vysledek);
}
```